POSTGRESQL INSTALLATION GUIDE

Remote KTH-access **2**
Connecting with Windows 2
Connecting with Mac/Linux: 3
Retrieve the password for your database account. 3
Accessing the database 4
Collaborating with your partner on the same Database 5
Creating several databases **6**
Moving files local → KTH-remote **6**
Solution 1 6
A couple of things to note here: 7
Solution 2 8
Populating the KTH-remote database with sample datasets 8
Connecting VPN **9**
Useful commands while exploring the database **10**

POSTGRESQL INSTALLATION GUIDE

Remote KTH-access

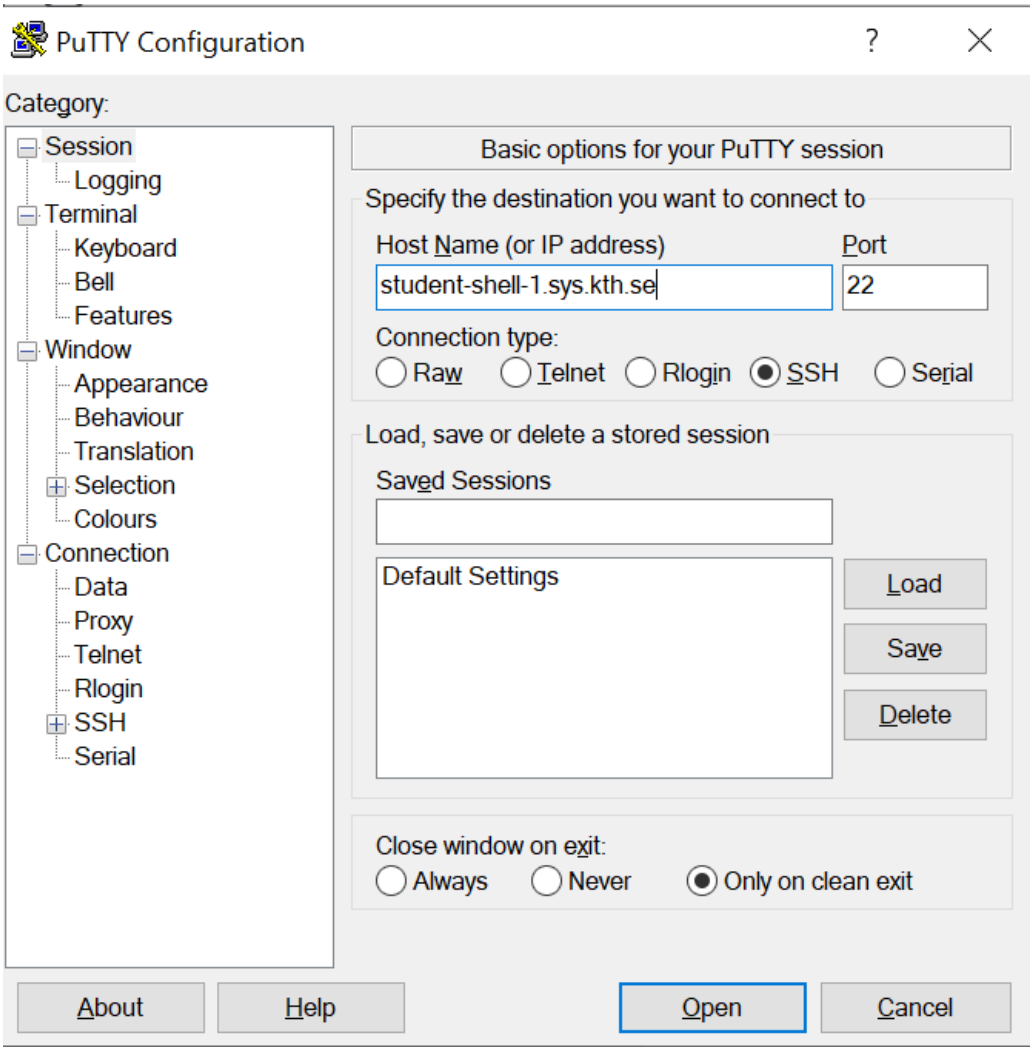
This is the recommended method to access the KTH remote server.

Connecting with Windows

Download and install the latest version of Putty. Please follow the following instructions for installing and configuring Putty.

Start Putty and add the Host IP-address of the KTH server:
`student-shell-1.sys.kth.se`, see the below figure for reference.

*Note: You can connect using the terminal. Staff-shell is required for employed students.



POSTGRESQL INSTALLATION GUIDE

Then press “Open” and you should be sent to a terminal in your home folder of the KTH-server.

Connecting with Mac/Linux:

We will be accessing KTHs server with SSH. The only thing you have to do is to login with your kthid and kth-password (i.e the first part of your kth-email).

```
$ ssh <kth username>@student-shell-1.sys.kth.se
```

You might get a warning that KTHs authenticity cannot be verified. In that case you just accept the connection regardless by writing **`yes`**.

You might need to write your kth-password in order to connect. Please note that no chars will be shown when you write your password! Not even stars (*)!

If everything is successful you should be in a new terminal environment!

Retrieve the password for your database account.

In order to be able to access the database environment, you will need a password. In order to retrieve the password you first navigate to the following directory:

```
$ cd Private/
```

When you are in the directory **`Private`**/, there should be a “password file”. Depending on which db you have access to[1] the file will have a name similar to **`.psqlpw-psql-dd1368-ht25.sys.kth.se`**

Make sure that there exists a password file in this directory (~/Private):

```
$ ls -lah
```

In this case the file that I (the author) have is called **`.psqlpw-psql-dd1368-ht25.sys.kth.se`**. This means that from here on out images and password-file references will be for this file. **So keep in mind which filename you have, and make the appropriate consideration when reading!**

IN CASE OF NO PASSWORD FILE

[1] This is a bit vague, like which db do I have access to? Well the IT department creates new databases for this course each year. And this is an effort to keep this guide up-to-date and reduce the miscommunication after each course instance.

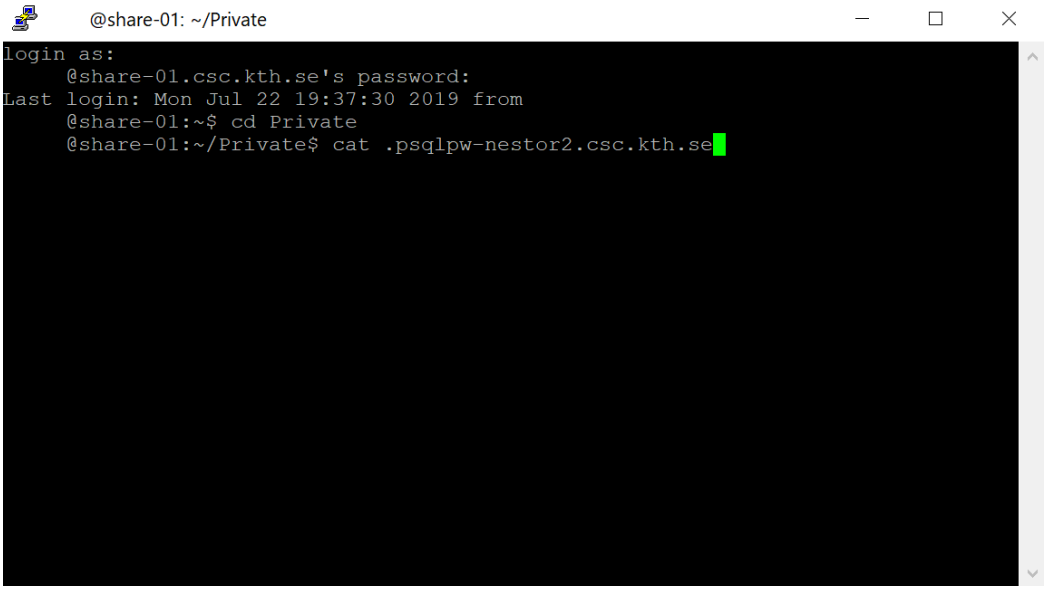
POSTGRESQL INSTALLATION GUIDE

If there is no password file, you do not have a database account and need to contact IT support and have them set you up with one.

Email template to IT-support (it-support@kth.se) in swedish:
Jag går Dena Hussains databaskurs, DD1368. Jag saknar databaskonto på servern psql-dd1368-ht25 (se KTH-INC-3703312). Kan ni skapa ett konto åt mig? Jag vet att ni lägger lösenordet i min AFS-hemkatalog (~/Private/.psqlpw-{servernamn}) och min lärare har förklarat hur man listar och visar innehållet i dolda filer.

IN CASE OF PASSWORD FILE
If that file exists, you can print the contents of the file to retrieve the password. Please write the following command:

```
$ cat .psqlpw-psql-dd1368-ht25.sys.kth.se
```



Accessing the database

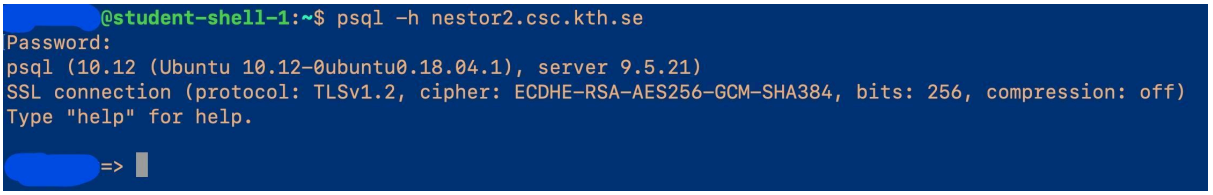
If everything is working at this stage, you should be able to start the database with the following command[2]:

```
$ psql -h psql-dd1368-ht25.sys.kth.se
```

If that command fails please restart your terminal and redo the KTH-ssh (or Putty) connection again.
If the command is successful you will be prompted to write the password that was in the file **`.psqlpw-psql-dd1368-ht25.sys.kth.se`**. Again, remember that when you write the password you will not get any chars not even stars (*)!

[2] Remember, this depends on which host you have access to! That is, what the name of your "password-file" is.

POSTGRESQL INSTALLATION GUIDE



Collaborating with your partner on the same Database

With your lab partner, you have to decide on who should be hosting the database. The partner that does not host the database will have to connect to the hosts' database every time they want to access it using the following commands:

```
$
    psql -h psql-dd1368-ht25.sys.kth.se

    Password: <contents of .psqlpw-psql-dd1368-ht25.sys.kth.se>

    mydb=>\c ‹insert the host user name here>
```

Both you (the host) and your partner will be able to create tables in the database. But your partner will not have automatic access to your created tables. In order to give your partner read and write privileges on a table perform the following Query when connected to the database:

```
dd1368=>  GRANT ALL PRIVILEGES ON TABLE <insert table name here>
          TO <insert partner username here>;
```

(Remember the semicolon (;) on the end!)

That is it! You have now established a connection to the databases located on KTH's servers! When you want to stop the connection you simply write:
mydb=>

```
mydb=>  \q                          # Exit psql with a psql-command

mydb=>  Ctrl+D                      # or with the shortcut
```

Then you can exit the ssh-connection by writing:

```
$ exit                              # Exit the ssh-connection
```

POSTGRESQL INSTALLATION GUIDE

```
$ Ctrl+D                              # or with the shortcut
```

(Otherwise, you can always just close the terminal window)

Creating several databases

Sometimes you want to have more than just one database ready at a time. For this, you can create new databases and use them in parallel.
To do this write:

```
$ CREATE DATABASE nameofdb;           # Creates the database

$ \c nameofdb                         # move to your new database
```

To switch between databases you only need the "\c" command and write the name of the database. Be careful however and remember the names of your databases so you don't lose them and fill the server with garbage. In the worst case you could use the "\l" command to list all available databases. This will however show hundreds of db's as it counts in all students and not just you.

Moving files local → KTH-remote

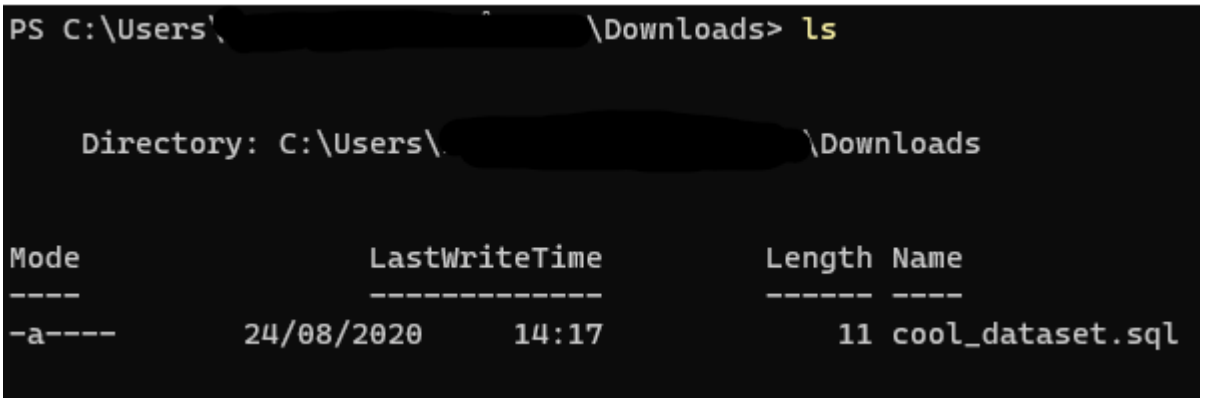
Sometimes you will have to populate your database with some sample data. For instance a mondial-sample dataset or maybe just your own **`cool_dataset.sql`**. However, how do you populate the KTH-remote database servers with your dataset?

Solution 1

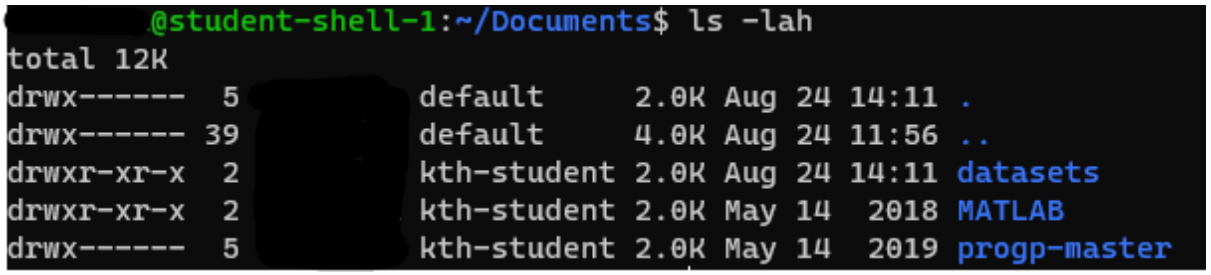
One of the easiest ways of moving files is with scp, you can also find out more about this in the KTH-scp guide. This command makes secure file copies between computers. The syntax might be a bit confusing so we will cover the details here.

Starting off, say that you have a file called **`cool_dataset.sql`** in your **`Downloads/`** directory:

POSTGRESQL INSTALLATION GUIDE



Then say that you have some directory on the KTH-remote called `~/Documents/datasets`:



In order to move the **cool_dataset.sql** to the above KTH-remote directory, we issue the following command:

```
$ scp ~/Downloads/cool_dataset.sql
  <kth-id>@student-shell-1.sys.kth.se:~/Documents/datasets
```

A couple of things to note here:

- "~" indicates your home directory. You could just, and might actually need to, expand this reference explicitly. So what you can do is change "~" with the result of calling:

```
$ cd ~/ && pwd                          # for Unix

$ cd ~/; pwd                            # for Windows
```

- Change `<kth-username>` to your personal kth-username.
- You will be prompted to write your KTH password.

If successful you will see a progress bar and status of the upload/copy:

POSTGRESQL INSTALLATION GUIDE

```
PS C:\Users\                    > scp 'C:\Users                    \Downloads\cool_dataset.sql'
        @student-shell-1.sys.kth.se:~/Documents/datasets/
        @student-shell-1.sys.kth.se's password:
cool_dataset.sql                                                    100%   11
    2.2KB/s   00:00
```

That is it! Then you can `ssh` or Putty into the KTH-remote servers and find the files at the destination:

```
Welcome to Ubuntu 18.04.5 LTS (GNU/Linux 4.15.0-112-generic x86_64)
Last login: Mon Aug 24 14:30:58 2020 from 85.230.83.208
        @student-shell-1:~$ ls -lah Documents/datasets/
total 5.0K
drwxr-xr-x 2          kth-student 2.0K Aug 24 14:32 .
drwx------ 5          default     2.0K Aug 24 14:11 ..
-rw-r--r-- 1          kth-student   11 Aug 24 14:32 cool_dataset.sql
        @student-shell-1:~$
```

Solution 2

Another more crude solution is creating a github repo, uploading everything to it and then just cloning it on the KTH servers. Some consider this a simpler solution as it is something they are more familiar with.

Populating the KTH-remote database with sample datasets

Now that you have moved your `cool_dataset.sql` to the KTH-remote servers, we can start populating the database! First thing, locate the file that you want to populate, in our case it is the following file, **`~/Documents/datasets/cool_dataset.sql`**, with the content:

```
CREATE TABLE account(

  accountNumber INT NOT NULL PRIMARY KEY,
  name VARCHAR(255) NOT NULL UNIQUE,
  password VARCHAR(255) NOT NULL,
  address VARCHAR(255) NOT NULL,
  joined DATE NOT NULL,
  expires DATE NOT NULL,
  paid DATE,
  bill INT NOT NULL
);
```

The most straight-forward approach is to pass the flag "**`-f <filename>`**" to the `**psql**` command like so:

POSTGRESQL INSTALLATION GUIDE

```
$ psql -h psql-dd1368-ht25.sys.kth.se -f <filename>
```

This will read the passed file as commands to be invoked on the database, which means that the file needs to be syntactically correct. In my case invoke the following:

```
[redacted]@student-shell-1:~$ psql -h nestor2.csc.kth.se -f ~/Documents/datasets/cool_dataset.sql
Password:
CREATE TABLE
```

(Where the password is the contents of ***`Private/.psqlpw-psql-dd1368-ht25.sys.kth.se`****)*

We can check that the table actually got created by passing **`\dt`** when logged in to the db:

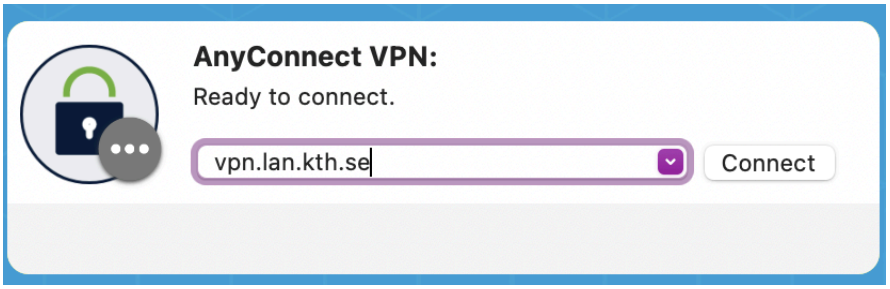
```
psql (10.12 (Ubuntu 10.12-0ubuntu0.18.04.1), server 9.5.21)
SSL connection (protocol: TLSv1.2, cipher: ECDHE-RSA-AES256-GCM-SHA384, bits: 256, compression: off)
Type "help" for help.

[redacted]=> \dt
          List of relations
 Schema |  Name   | Type  |  Owner
--------+---------+-------+----------
 public | account | table | [redacted]
(1 row)
```

Another way of populating the data is to log into the database and then call ***`\i <filename>`****. It is the same idea as previously (you provide a syntactically correct .sql-file), but now you are already logged in to the database.*

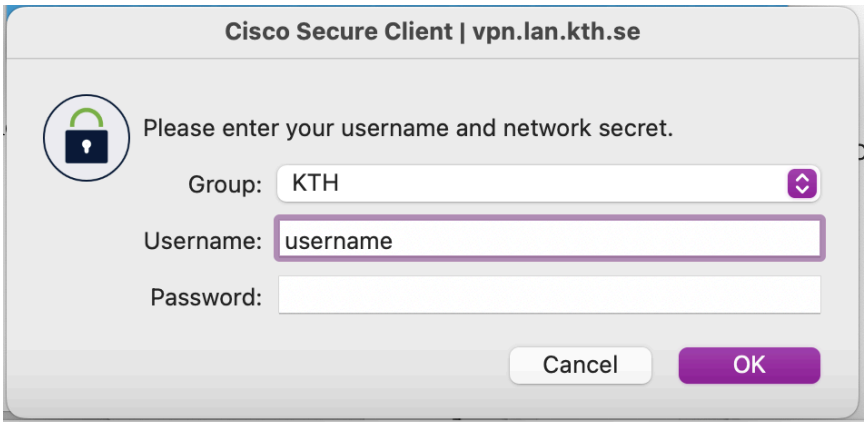
Connecting VPN

Follow this guide to connect to KTH's network.
https://intra.kth.se/en/it/arbeta-pa-distans/vpn/vpn-pa-kth-1.776625
To connect write the following: vpn.lan.kth.se



After this you will get this you will get a prompt asking for your username and password:

POSTGRESQL INSTALLATION GUIDE



Your username is not username@kth.se, just username.
The password can be found here network secret (same as for eduroam)

Useful commands while exploring the database

`\?` - Shows all commands available
`\q` - Quit psql.
`\dt` - Shows all current tables in your database and their owners.
`\dp` - Shows all current tables in your database and the access privileges.
`\timing` - Shows query execution time while toggled on.
`\x` - Show extended view, good for big tables
`\i` <filename> - Reads and executes the contents of a specific file.
`\c` <database name> -switch database
`\l` -list all databases in the server

You can peruse the full list of commands by typing `\h`, and while looking at and navigating output you can press `h` (while in output view) to see the full list of navigation commands. Hint it's similar to Vi editor.